SOFTWARE REQUIREMENTS SPECIFICATION

CONVOYSYNC

Smart Group Trip Coordination Platform

Document field	Value
Document identifier	CS-SRS-001
Version	1.0
Date	20 September 2026
Course	IT-314 — Software Engineering
Group	Group 8
Document status	Software Requirements Specification
Prepared for	software development

Prepared by Group 8

Document Control

Version	Date	Description	Prepared by
1.0	20 September 2026	Initial consolidated SRS for ConvoySync based on available project requirements and sprint records.	Group 8

Review item	Status / note
Project	ConvoySync — Smart Group Trip Coordination Platform
Scope baseline	Product-level target requirements; Sprint 1 boundaries are identified separately.
Implementation claims	Requirements in this document define intended behaviour and do not imply that every feature has already been implemented or tested.
Approval	Approve by all team members.

Contents

- 1. Introduction
- 2. Overall Description
- 3. System Features and Functional Requirements
- 4. External Interface Requirements
- 5. Data Requirements
- 6. Non-Functional Requirements
- 7. Business Rules and Validation
- 8. Use Cases
- 9. Requirements Traceability and Verification
- 10. Scope Boundaries and Future Enhancements
- Appendix A. Glossary
- Appendix B. References

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional requirements, external interfaces, data requirements, quality attributes, constraints, and acceptance conditions for ConvoySync — Smart Group Trip Coordination Platform. It provides a shared baseline for stakeholders, developers, testers, and evaluators. The requirements are written so that they can guide implementation and be checked through inspection, demonstration, analysis, or testing.

1.2 Scope

ConvoySync is a web-based platform intended to help people coordinate group trips and maintain a shared record of trip-related expenses. Its core product capabilities are account registration and login, access-controlled use of the platform, trip creation and joining through a code or link, and recording and distributing trip expenses among members.

The product is intended to reduce coordination effort, make expense shares visible to trip participants, and provide a common place to review trip and expense information.

The current product scope excludes payment-gateway processing. Live location sharing, route monitoring, emergency/SOS alerts, convoy-separation handling, RSVP/commitment tracking, privacy/ghost mode, adaptive GPS/battery behaviour, and administrator analytics are treated as deferred or future capabilities rather than assumed Sprint 1 deliverables.

1.3 Intended Audience

Audience	Use of this document
Project stakeholders	Review the intended scope and confirm requirements.
Developers	Use requirements as the basis for design and implementation.
Testers / quality assurance	Derive test cases and verify observable outcomes.
Project supervisor / evaluator	Assess requirement completeness, consistency, and traceability.
Future maintainers	Understand expected behaviour and documented constraints.

1.4 Product Scope and Release Interpretation

This SRS specifies the intended product behaviour at the product level. The Sprint 1 records describe a planned increment comprising Authentication and Home, Trip Generation and Joining, and Expense Management. A requirement may therefore be in scope for the overall product without being confirmed as implemented in Sprint 1. Implementation and verification status must be established from code, tests, and review evidence, not inferred from the presence of a requirement.

1.5 Definitions, Acronyms, and Abbreviations

Term	Definition
SRS	Software Requirements Specification.
Trip Host	User who creates or manages a trip and coordinates its participants.
Trip Member	User who joins a trip and can access trip information according to permissions.
Administrator	Privileged operational role, if enabled; administrative analytics are deferred from Sprint 1.
Expense	A monetary cost recorded against a trip.
Expense share	The portion of an expense assigned to a participant.
Custom split	A host-defined allocation of an expense that need not be an equal split.
Authentication	Verification of a user's identity.
Authorization	Determination of which resources or actions an authenticated user may access.
JWT	JSON Web Token, used by the described application stack for token-based authentication.
API	Application Programming Interface.
NFR	Non-Functional Requirement.
FR	Functional Requirement.

1.6 Document Conventions

- “Shall” indicates a mandatory requirement.
- Each requirement has a unique identifier to support review and traceability.
- Priority “Must” indicates core behaviour needed for the defined product scope; “Should” indicates desirable behaviour that may be scheduled after core behaviour.
- Acceptance criteria describe observable pass/fail outcomes. Any numeric quality target is a proposed project target and should be confirmed against course expectations and actual deployment constraints.

1.7 References

The document structure and requirement-writing approach are informed by ISO/IEC/IEEE 29148:2018, Systems and software engineering — Life cycle processes — Requirements engineering. This document is a project-specific SRS; it does not claim formal certification or full compliance with the standard.

Project source basis: ConvoySync requirements and user-stories material, product backlog / sprint mapping and traceability material, Sprint 1 report records available for the project. Where the records distinguish planned scope from implementation evidence, this SRS preserves that distinction.

2. Overall Description

2.1 Product Perspective

ConvoySync is a browser-accessible client-server web application. The described implementation stack uses React 18 and Vite for the frontend, Python/FastAPI for backend API services, PostgreSQL 16 for persistent storage, JWT for authentication, and SQLAlchemy for database access. These technologies describe the current project context; the SRS primarily specifies required externally observable behaviour.

2.2 Product Functions

- Create a user account and authenticate registered users.
- Restrict protected features and data to appropriately authenticated and authorized users.
- Provide a home page with navigation to the principal product areas.
- Create trips and generate a unique joining code or link.
- Allow eligible users to join trips using a valid code or link.
- Record expenses within a trip and associate each expense with its creator and trip.
- Allocate an expense equally or through a custom split, and show each participant's share.
- Display trip expenses and share information to authorized trip participants.
- Provide clear feedback for invalid input, unavailable operations, and failed requests.

2.3 User Classes and Characteristics

User class	Goals	Permissions / concerns
Visitor	Understand the service and create an account or sign in.	May access public entry pages only; cannot view protected trip data.
Registered user	Access the platform and manage or join trips.	May access only their own account and trips for which they have permission.
Trip Host	Create a trip, invite members, record expenses, and manage expense allocation.	Host-only trip actions must be restricted to the host or explicitly authorized users.
Trip Member	Join a trip and review relevant trip and expense information.	Must not modify another trip or data outside their membership permissions.
Administrator	Potential future operational oversight.	Admin analytics and broad cross-trip access are not part of the Sprint 1 commitment.

2.4 Operating Environment

- Client: a modern web browser on a desktop, laptop, tablet, or mobile device.
- Frontend: React 18 application built and served using Vite.
- Backend: Python application exposing HTTP API endpoints through FastAPI.
- Database: PostgreSQL 16, accessed through SQLAlchemy.
- Authentication: token-based authentication using JWT as described in the project stack.
- Network: client and server communicate over HTTP-based APIs; production deployment should use HTTPS.

2.5 Design and Implementation Constraints

- The project’s current stack is React, Vite, FastAPI, PostgreSQL, JWT, and SQLAlchemy.
- Trip and expense information must be persisted rather than existing only in the browser session.
- Protected operations must enforce authorization on the server; hiding a button in the frontend is not sufficient.
- No payment gateway is included in the defined scope.

2.6 Assumptions and Dependencies

ID	Assumption / dependency	Impact if invalid
ASM-01	Users have access to a supported browser and a network connection to the service.	Users may be unable to access the web application.

ASM-02	A user account is required for protected trip and expense features.	Access-control and data ownership rules would need revision.
DEP-01	The backend API and PostgreSQL database are available for normal operation.	Create, join, and expense operations may fail or be delayed.
DEP-02	The project team will define exact field-level validation and currency rules before implementation is finalized.	Ambiguous input or monetary calculations may produce inconsistent results.

2.7 Product Boundaries

Included in the product requirements	Excluded or deferred from Sprint 1
Registration, login, authorization, home/navigation	Live location sharing and route monitoring
Trip creation and joining by code/link	Emergency/SOS alerts
Expense recording, equal/custom split, expense/share tracking	Convoy separation and adaptive GPS/battery behaviour
Trip-member access control and error feedback	RSVP/commitment tracking and privacy/ghost mode
No payment gateway assumed	Administrator analytics across trips

3. System Features and Functional Requirements

The following requirements describe intended system behaviour. They are not assertions that each feature is already implemented. Unless stated otherwise, verification is by test.

3.1 Authentication and Account Access

FR-AUTH-01 — Account registration

Requirement: The system shall allow a visitor to submit the required registration details and create an account when all required fields are valid and the account identifier is not already in use.

Priority: Must | Verification: Test

Acceptance criteria: A valid submission creates one account and produces a success response; invalid or duplicate details do not create an account.

FR-AUTH-02 — User login

Requirement: The system shall allow a registered user to submit login credentials and shall establish an authenticated session only when the credentials are valid.

Priority: Must | Verification: Test

Acceptance criteria: Valid credentials grant access to protected pages; invalid credentials do not grant access and produce a clear error.

FR-AUTH-03 — Authenticated-user retrieval

Requirement: The system shall provide a protected operation for the client to retrieve the identity and permitted profile information of the currently authenticated user.

Priority: Must | Verification: Test

Acceptance criteria: A valid authenticated request returns the current user's profile; an unauthenticated or invalid-token request is rejected.

FR-AUTH-04 — Logout

Requirement: The client shall provide a logout action that clears the active client-side authentication state and returns the user to an unauthenticated view.

Priority: Must | Verification: Test

Acceptance criteria: After logout, protected client views are no longer available without authenticating again.

FR-AUTH-05 — Protected navigation

Requirement: The system shall restrict protected trip and expense features to authenticated users and shall enforce access control on the backend for protected API operations.

Priority: Must | Verification: Test

Acceptance criteria: Direct API requests without valid authorization are rejected; frontend-only restrictions are not considered sufficient.

FR-AUTH-06 — Input validation

Requirement: The system shall validate required registration and login fields and display actionable feedback when a submission is incomplete or malformed.

Priority: Must | Verification: Test

Acceptance criteria: Missing or malformed required fields are identified without creating an invalid account or authenticated session.

3.2 Home and Navigation

FR-HOME-01 — Home page

Requirement: The system shall present an authenticated home page that provides navigation to the available trip and expense functions.

Priority: Must | Verification: Test

Acceptance criteria: After successful login, the user can reach the home page and see the available feature entry points.

FR-HOME-02 — Authentication-aware routing

Requirement: The client shall route unauthenticated users to an appropriate public or login view when they attempt to open a protected page.

Priority: Must | Verification: Test

Acceptance criteria: A protected page does not expose trip or expense data before authentication.

3.3 Trip Creation and Joining

FR-TRIP-01 — Create trip

Requirement: The system shall allow an authenticated user to create a trip by submitting the required trip details, including a trip name or title and destination where those fields are required by the selected trip form.

Priority: Must | Verification: Test

Acceptance criteria: A valid request creates one trip record associated with the creating user as host; invalid input is rejected with feedback.

FR-TRIP-02 — Assign trip host

Requirement: The system shall record the creating user as the trip host and shall associate that user with the trip's membership or ownership record.

Priority: Must | Verification: Test

Acceptance criteria: The trip creator is identifiable as host and has the host permissions defined for the trip.

FR-TRIP-03 — Unique joining code or link

Requirement: The system shall generate or provide a unique joining code or link for a trip that supports member joining.

Priority: Must | Verification: Test

Acceptance criteria: The code/link identifies the intended trip and does not ambiguously identify another active trip.

FR-TRIP-04 — Join by code

Requirement: The system shall allow an authenticated user to join a trip by submitting a valid joining code.

Priority: Must | Verification: Test

Acceptance criteria: A valid code adds the user to the corresponding trip; an invalid code produces an error and does not create membership.

FR-TRIP-05 — Join by link

Requirement: The system shall allow an authenticated user to open a valid trip joining link and complete the joining flow.

Priority: Must | Verification: Test

Acceptance criteria: A valid link leads to the intended trip's join flow; an invalid or unavailable link does not add the user to a trip.

FR-TRIP-06 — Trip membership

Requirement: The system shall maintain a relationship between a trip and each user who has joined it.

Priority: Must | Verification: Test

Acceptance criteria: Membership is persisted and can be used to authorize access to trip details and expenses.

FR-TRIP-07 — Trip access control

Requirement: The system shall permit trip details to be viewed or modified only by users with the required permissions for that trip.

Priority: Must | Verification: Test

Acceptance criteria: A non-member cannot retrieve private trip information by guessing an identifier; a member can access permitted trip information.

FR-TRIP-08 — Duplicate join handling

Requirement: The system shall handle an attempt by an existing member to join the same trip again without creating duplicate membership records.

Priority: Should | Verification: Test

Acceptance criteria: The system returns an understandable result and retains only one active membership for that user and trip.

3.4 Expense Management

FR-EXP-01 — Record expense

Requirement: The system shall allow an authorized trip participant, subject to trip permissions, to record an expense against a trip with a description, amount, date, and payer or creator information.

Priority: Must | Verification: Test

Acceptance criteria: A valid expense is saved against the correct trip; missing, malformed, or non-positive amounts are rejected.

FR-EXP-02 — Expense ownership and association

Requirement: The system shall associate each expense with one trip and record the user who created it; where the payer differs from the creator, the payer shall be represented separately.

Priority: Must | Verification: Test

Acceptance criteria: An expense can be traced to its trip and creator, and the payer is not inferred incorrectly when explicitly provided.

FR-EXP-03 — Equal split

Requirement: The system shall support allocating an expense equally among a selected set of eligible trip members.

Priority: Must | Verification: Test

Acceptance criteria: The sum of allocated shares equals the expense total to the currency's supported precision; any rounding remainder is handled consistently.

FR-EXP-04 — Custom split

Requirement: The system shall support a host-authorized custom allocation of an expense among selected eligible trip members.

Priority: Must | Verification: Test

Acceptance criteria: The system validates the custom allocation and rejects it if the shares do not reconcile with the expense total under the defined currency precision.

FR-EXP-05 — Share calculation and persistence

Requirement: The system shall calculate or validate each participant's share and persist the resulting allocation with the associated expense.

Priority: Must | Verification: Test

Acceptance criteria: Reloading the expense shows the same saved shares; share totals reconcile to the expense amount.

FR-EXP-06 — Expense list

Requirement: The system shall display expenses belonging to a trip to users who are authorized members of that trip.

Priority: Must | Verification: Test

Acceptance criteria: The list contains only expenses for the selected trip that the current user is allowed to view.

FR-EXP-07 — Share visibility

Requirement: The system shall show the relevant expense amount and participant share allocation to authorized trip participants.

Priority: Must | Verification: Test

Acceptance criteria: Displayed amounts and shares match persisted records and are associated with the correct participants.

FR-EXP-08 — Expense validation

Requirement: The system shall validate the expense amount, required description, trip association, selected participants, and split totals before saving an expense.

Priority: Must | Verification: Test

Acceptance criteria: Invalid expense data is not persisted and the user receives feedback identifying the validation failure.

FR-EXP-09 — Expense nudging

Requirement: If expense reminders or nudges are enabled in the delivered release, the system shall provide a user-visible prompt related to an outstanding expense share without representing the prompt as a completed payment.

Priority: Should | Verification: Test

Acceptance criteria: A nudge identifies the relevant expense/share and does not claim that money was transferred.

FR-EXP-10 — Expense access control

Requirement: The system shall prevent users who are not authorized for a trip from creating, changing, or retrieving that trip's expense records.

Priority: Must | Verification: Test

Acceptance criteria: Unauthorized read and write attempts are rejected and do not change stored data.

3.5 Error Handling and System Feedback

FR-ERR-01 — Actionable error messages

Requirement: The system shall provide a clear message when a user action fails because of invalid input, missing authorization, an unavailable resource, or a server/network failure.

Priority: Must | Verification: Test

Acceptance criteria: The user receives a meaningful message and the interface remains usable after a recoverable failure.

FR-ERR-02 — No false success

Requirement: The system shall show a success confirmation only after the corresponding operation has succeeded and any required persistence has completed.

Priority: Must | Verification: Test

Acceptance criteria: Failed create/join/expense requests are not presented as successful.

FR-ERR-03 — Retry safety

Requirement: The system should avoid creating duplicate trips, memberships, or expenses when a user retries after an ambiguous network response, using suitable request handling or duplicate detection.

Priority: Should | Verification: Test

Acceptance criteria: Repeated requests are handled according to the operation's duplicate policy and do not silently create unintended duplicate records.

4. External Interface Requirements

4.1 User Interface

- The interface shall provide distinct public authentication views and authenticated application views.
- Forms shall identify required fields and display validation feedback near the relevant action or field.
- The home page shall provide navigation to trip and expense functions that are available in the deployed release.
- The interface shall distinguish loading, success, empty, and error states for operations that retrieve or change data.
- The interface shall not expose sensitive trip information in public pages or error messages.
- Layout should remain usable on common desktop and mobile browser widths; exact supported breakpoints should be confirmed during UI implementation.

4.2 Software Interfaces

Interface ID	Interface	Purpose / requirements
IF-SW-01	React/Vite frontend to FastAPI backend	Exchange request and response data for authentication, trips, membership, and expenses through documented HTTP API endpoints.
IF-SW-02	FastAPI backend to PostgreSQL	Persist users, trips, membership, expenses, and expense shares with appropriate integrity constraints.
IF-SW-03	Backend authentication middleware / JWT	Validate authentication tokens and derive the current user identity for protected operations.
IF-SW-04	SQLAlchemy data-access layer	Map application operations to database access and transactions.

Exact endpoint paths, request schemas, response schemas, and HTTP status codes are implementation interface details and must be documented consistently in the API specification or code. This SRS does not invent endpoint URLs that are not confirmed by the project.

4.3 Communication Interfaces

- The browser and backend shall exchange structured request and response data over HTTP-based APIs.
- Production deployments shall protect credentials and authentication tokens in transit using HTTPS.
- The client shall handle API errors and network interruptions without falsely confirming an unsuccessful operation.
- The backend shall validate all client-supplied values; client-side validation alone shall not be trusted.

4.4 Hardware Interfaces

No dedicated hardware interface is required for the current web-based scope. The system relies on standard user computing devices and network connectivity.

4.5 External Services

No payment gateway or live-location provider is required for the current defined scope. If such services are introduced in a future release, their interfaces, privacy implications, failure behaviour, and security requirements must be specified separately.

5. Data Requirements

5.1 Core Data Entities

Entity	Required information (logical level)	Key relationships
User	Unique user identifier; registration/login identity; required profile fields; account timestamps/status as needed.	May host trips, join trips, and create expenses.
Trip	Unique trip identifier; title/name; destination or trip details; host; joining code/link; creation/update information.	Has one host and multiple members; contains expenses.
Trip Membership	Trip identifier; user identifier; membership role/status; join timestamp.	Associates users with trips and supports access control.
Expense	Unique expense identifier; trip; description; amount; currency; expense date; creator; payer where applicable.	Belongs to one trip and has one or more shares.
Expense Share	Expense identifier; participant; allocated amount; share/settlement status if implemented.	Associates an expense with a participant and stores the participant's allocation.

5.2 Data Integrity Rules

- Each user, trip, and expense shall have a unique primary identifier.
- A trip shall reference a valid host user.
- A membership shall reference an existing user and an existing trip; duplicate active memberships for the same user and trip shall be prevented.
- An expense shall reference an existing trip and a valid creator; participant shares shall reference valid eligible participants.
- An expense amount and each allocated share shall be stored using a numeric representation suitable for monetary values, not binary floating-point arithmetic.
- For a fully allocated expense, the sum of participant shares shall equal the expense total to the supported currency precision.
- Database transactions shall preserve consistency when an expense and its shares are saved.

5.3 Data Validation

- Required text fields shall not be accepted as empty or whitespace-only values.
- Amounts shall be numeric, positive where required, and within the supported precision and range.
- A joining code/link shall resolve to an existing joinable trip.
- A split shall not include users who are not eligible participants in the relevant trip.
- Server-side validation shall be applied regardless of whether the client performs the same checks.

5.4 Data Retention and Deletion

The available project requirements do not define a specific retention period, account deletion policy, or archival schedule. These policies must be agreed with stakeholders before production deployment. Until defined, the implementation should avoid deleting records in a way that silently breaks the integrity of trips, expenses, or share history.

6. Non-Functional Requirements

The following requirements are proposed quality targets for the project. Where a numeric target is not already established by the project records, it should be confirmed by the team and supervisor before it is treated as a contractual acceptance threshold.

6.1 Security and Privacy

NFR-SEC-01 — Authentication enforcement

Requirement: The system shall authenticate protected requests and reject requests with missing, invalid, or expired authentication credentials.

Priority: Must | Verification: Test

Acceptance criteria: Automated tests verify that protected endpoints reject unauthenticated requests.

NFR-SEC-02 — Authorization enforcement

Requirement: The backend shall check user permissions for trip and expense resources on every protected operation.

Priority: Must | Verification: Test

Acceptance criteria: Tests using a non-member or unauthorized user cannot read or modify another trip's protected records.

NFR-SEC-03 — Credential protection

Requirement: The system shall not store user passwords in plaintext; passwords shall be handled using an established password-hashing approach if password-based registration is used.

Priority: Must | Verification: Test

Acceptance criteria: Inspection confirms password storage is hashed and no plaintext password is returned by an API.

NFR-SEC-04 — Transport security

Requirement: Production access to the application shall use HTTPS to protect credentials and authentication tokens in transit.

Priority: Must | Verification: Test

Acceptance criteria: Deployment configuration redirects or otherwise prevents sensitive production traffic over unprotected HTTP.

NFR-SEC-05 — Sensitive error handling

Requirement: The system shall not expose passwords, secret keys, raw authentication tokens, or internal stack traces in user-facing error messages.

Priority: Must | Verification: Test

Acceptance criteria: Error responses contain safe messages and do not disclose secrets or internal implementation details.

NFR-PRIV-01 — Trip data privacy

Requirement: The system shall limit access to trip and expense data to users with appropriate membership or host permissions.

Priority: Must | Verification: Test

Acceptance criteria: Access-control tests cover both permitted and denied access.

6.2 Performance

NFR-PERF-01 — Common-operation responsiveness

Requirement: Under the agreed test environment and normal project demonstration load, the system should return a response for common read operations such as loading the home page's trip information within 3 seconds at the 95th percentile, excluding initial asset download and external network delay.

Priority: Should | Verification: Performance test

Acceptance criteria: A documented performance test records the environment, sample size, and percentile result; target is confirmed before formal acceptance.

NFR-PERF-02 — Write-operation responsiveness

Requirement: Under the same agreed test conditions, valid trip and expense write operations should normally complete within 3 seconds at the 95th percentile.

Priority: Should | Verification: Performance test

Acceptance criteria: Performance testing records the completion time of valid create/join/expense requests; target is confirmed before formal acceptance.

6.3 Reliability and Data Integrity

NFR-REL-01 — Persistence

Requirement: The system shall persist successfully created users, trips, memberships, expenses, and shares in the database so that they remain available after a normal page refresh or new login.

Priority: Must | Verification: Test

Acceptance criteria: Saved records can be retrieved in a later authenticated session.

NFR-REL-02 — Transactional consistency

Requirement: The system shall prevent partially saved expense allocations when the expense and its shares must be committed as one logical operation.

Priority: Must | Verification: Test

Acceptance criteria: A forced failure during a save leaves either the complete valid operation or no partial allocation.

NFR-REL-03 — Failure recovery feedback

Requirement: When a request fails, the system shall preserve user-entered values where feasible and provide a safe way to retry or return to a stable view.

Priority: Should | Verification: Test

Acceptance criteria: A simulated request failure does not falsely report success and the user can recover without unintended duplicate data.

6.4 Usability and Accessibility

NFR-USE-01 — Understandable interaction

Requirement: The system shall label primary actions, required fields, and validation errors using clear and consistent language.

Priority: Must | Verification: Inspection / demonstration

Acceptance criteria: A representative user can identify how to register, log in, create/join a trip, and record an expense without interpreting developer terminology.

NFR-USE-02 — Responsive layout

Requirement: The interface should remain usable on supported desktop and mobile browser widths without essential controls being inaccessible due to horizontal overflow.

Priority: Should | Verification: Inspection

Acceptance criteria: Representative pages are inspected at the team's agreed desktop and mobile viewport sizes.

NFR-USE-03 — Accessible form controls

Requirement: Form fields and actions should have programmatically associated labels, keyboard-operable controls, and visible focus indication.

Priority: Should | Verification: Inspection

Acceptance criteria: Keyboard-only navigation can reach and operate primary form controls, and labels can be identified by assistive technology.

6.5 Maintainability and Compatibility

NFR-MAIN-01 — Separation of concerns

Requirement: The implementation should keep presentation, API handling, business validation, and persistence responsibilities sufficiently separated to support maintenance and testing.

Priority: Should | Verification: Inspection

Acceptance criteria: Code review can identify the frontend, backend service/API, and persistence responsibilities.

NFR-COMP-01 — Browser compatibility

Requirement: The web application shall support current stable versions of commonly used modern browsers selected by the project team.

Priority: Should | Verification: Test

Acceptance criteria: The team documents the browser versions used for acceptance testing and verifies primary flows in those browsers.

7. Business Rules and Validation

Rule ID	Rule
BR-01	A protected trip or expense operation requires a valid authenticated user.
BR-02	The trip creator is recorded as the trip host.
BR-03	Only a valid joining code/link may add a user to the corresponding trip.
BR-04	A user cannot create duplicate active membership records for the same trip.
BR-05	Trip membership and host permissions determine access to trip and expense information.
BR-06	An expense must belong to one existing trip.
BR-07	The sum of allocated shares must reconcile with the expense total to the supported currency precision.
BR-08	A failed operation must not be presented as successful.
BR-09	Expense nudges, if present, are reminders only and do not represent payment processing.
BR-10	Payment-gateway transactions are outside the current defined scope.

7.1 Monetary Calculation Policy

The implementation shall use decimal-safe monetary storage and calculations. For equal splits, any remainder caused by currency precision shall be distributed using one documented, deterministic rule so that the total of shares equals the expense total. For custom splits, the system shall define whether the host enters fixed amounts or percentages

and validate the allocation accordingly. The exact supported currencies and rounding policy must be confirmed before release; this SRS does not assume a payment currency or settlement service.

7.2 Authorization Policy

The backend is the authority for access decisions. The frontend may hide or disable unavailable actions to improve usability, but this shall not replace backend checks. Denied operations must not reveal private records through response bodies, error messages, or predictable resource identifiers.

8. Use Cases

UC-01: Register and Log In

Field	Description
Primary actor	Visitor / Registered user
Preconditions	The web application is reachable.
Trigger	The visitor selects Register or the registered user selects Login.
Main flow	1. User opens the relevant form. 2. User enters required details. 3. System validates input. 4. For registration, system creates the account; for login, system validates credentials. 5. On success, the user is routed to the authenticated home page.
Alternate / exception flow	Invalid or missing data produces validation feedback. Duplicate registration is rejected. Invalid credentials do not authenticate the user. Network/server failure is shown without false success.
Postconditions	A successful registration creates one account; a successful login establishes authenticated client state.

UC-02: Create a Trip

Field	Description
Primary actor	Authenticated user; becomes Trip Host
Preconditions	User is authenticated.
Trigger	User selects Create Trip.
Main flow	1. System displays the trip form. 2. User enters required trip details. 3. System validates the details. 4. System creates the trip and records the host. 5. System provides the trip details and joining code/link.
Alternate / exception flow	Invalid details are rejected. If persistence fails, no success confirmation is shown and the user receives an error.
Postconditions	A trip exists, has a host, and has a usable joining mechanism.

UC-03: Join a Trip

Field	Description
Primary actor	Authenticated user
Preconditions	User is authenticated and has a joining code or link.
Trigger	User submits a code or opens a joining link.
Main flow	1. System resolves the code/link. 2. System verifies the trip is joinable. 3. System checks whether the user is already a member. 4. System creates membership if needed. 5. System confirms the result.
Alternate / exception flow	Invalid code/link is rejected. Existing membership is handled without duplication. A failed request does not create membership.
Postconditions	The user is a member of the intended trip, or no membership change occurs.

UC-04: Record and Split an Expense

Field	Description
Primary actor	Authorized trip participant; host for host-restricted split actions
Preconditions	User is authenticated and authorized for the selected trip.
Trigger	User selects Add Expense.
Main flow	1. User enters expense details. 2. User selects eligible participants and an equal or custom split. 3. System validates amount and allocation. 4. System saves the expense and shares consistently. 5. System displays the saved expense and share details.
Alternate / exception flow	Invalid amount, invalid participant, mismatched shares, unauthorized action, or persistence failure prevents successful completion and is reported.
Postconditions	A valid expense and its shares are persisted and visible to authorized users.

UC-05: View Trip Expenses

Field	Description
Primary actor	Authorized trip member / host
Preconditions	User is authenticated and has access to the trip.
Trigger	User opens a trip's expense view.
Main flow	1. System verifies membership/permission. 2. System retrieves expenses and share allocations for the trip. 3. System displays the records and any available empty state.
Alternate / exception flow	Unauthorized access is denied. Retrieval failure is shown as an error rather than an empty successful result.
Postconditions	The user sees only permitted expense and share information.

9. Requirements Traceability and Verification

The matrix below maps requirements to the main product areas and a suitable verification method. Detailed test cases should be maintained separately and linked to these identifiers.

Requirement group	Feature / objective	Verification evidence
FR-AUTH-01 to FR-AUTH-06	Registration, login, current-user retrieval, logout, authorization, validation	Functional tests; API authorization tests; code/configuration inspection
FR-HOME-01 to FR-HOME-02	Authenticated home page and protected routing	UI demonstration; route/access tests
FR-TRIP-01 to FR-TRIP-08	Trip creation, host assignment, code/link joining, membership and access control	Functional tests; database checks; negative authorization tests
FR-EXP-01 to FR-EXP-10	Expense creation, equal/custom split, share persistence, visibility and nudges	Functional tests; monetary reconciliation tests; database transaction tests
FR-ERR-01 to FR-ERR-03	Error feedback, truthful success states, retry handling	Negative tests; simulated network/server failures
NFR-SEC-01 to NFR-PRIV-01	Authentication, authorization, credential and trip-data protection	Security tests; configuration/code inspection
NFR-PERF-01 to NFR-PERF-02	Response-time targets	Repeatable performance test with documented environment
NFR-REL-01 to NFR-REL-03	Persistence, transaction consistency, recovery	Integration tests; failure injection
NFR-USE-01 to NFR-USE-03	Usability, responsive layout, accessible forms	Inspection; keyboard-only demonstration
NFR-MAIN-01 and NFR-COMP-01	Maintainability and browser compatibility	Code review; browser acceptance tests

9.1 Acceptance Test Scenarios

1. Register with valid details and verify that the account can be used to log in.
2. Attempt registration with missing required fields and verify that no account is created.
3. Log in with invalid credentials and verify that protected access is not granted.

4. Call a protected API without a valid token and verify that the request is rejected.
5. Create a trip and verify that its host and joining code/link are stored.
6. Join the trip with a valid code and verify membership; repeat the action and verify no duplicate membership.
7. Attempt to join with an invalid code and verify that membership is not created.
8. Attempt to access another trip as a non-member and verify that private data is not returned.
9. Create an expense with a valid amount and equal split; verify the shares sum to the expense total.
10. Create a custom split with mismatched totals and verify that it is rejected.
11. Simulate a failure during expense save and verify that partial shares are not left behind.
12. Refresh or sign in again and verify that previously saved trip and expense data remains available.
13. Simulate an API/network failure and verify that the UI does not show a false success message.

9.2 Sprint 1 Scope and Evidence Boundary

The documented Sprint 1 scope covers Authentication and Home, Trip Generation and Joining, and Expense Management, including equal/custom expense splitting, expense/share tracking, and nudging. Available project records indicate that authentication work was merged and that trip-module work had implementation evidence, while implementation and verification must be judged separately. README-level evidence specifically establishes registration, login, current-user retrieval, client-side logout, and authentication/home-page structure; it does not by itself establish that all trip and expense requirements are complete or tested.

Accordingly, this SRS describes the intended behaviour and acceptance conditions. It does not label all requirements as implemented, tested, or accepted.

10. Scope Boundaries and Future Enhancements

Capability	Treatment in this SRS
Payment gateway / actual money transfer	Out of scope. Expense tracking and reminders must not be represented as money transfer.
Live location sharing	Deferred future capability; no current location collection or sharing requirement is imposed.
Route monitoring / route optimization	Deferred future capability.
Emergency / SOS alerts	Deferred future capability; no emergency-response guarantee is made.
Convoy separation detection	Deferred future capability.
RSVP / commitment tracking	Deferred future capability.
Privacy / ghost mode	Deferred future capability requiring separate privacy requirements.
Adaptive GPS / battery behaviour	Deferred future capability.
Administrator cross-trip analytics	Deferred; not part of the Sprint 1 commitment.

Adding any deferred capability requires updated scope, requirements, interface and data definitions, privacy/security analysis where relevant, and new acceptance tests.

Appendix A. Glossary

Term	Meaning
Acceptance criterion	Observable condition used to decide whether a requirement has been satisfied.
Custom split	An expense allocation configured by the host rather than divided equally.
Joining code/link	A trip-specific value or URL used to enter a trip's join flow.
Traceability	Ability to relate a requirement to its source, implementation area, and verification evidence.
Sprint	A time-boxed development iteration in an Agile workflow.

Appendix B. References

14. ISO/IEC/IEEE 29148:2018, Systems and software engineering — Life cycle processes — Requirements engineering.
15. ConvoySync project requirements and user-stories document, product backlog / sprint mapping, requirements traceability material, Sprint 1 report records available in the project context.